

Through the Object-Centric Looking Glass: a Mereotopology-based Foundation for Transparent Event Log Abstraction and Case Extraction

RILEY MOHER[✉] and MICHAEL GRUNINGER, University of Toronto, Canada

Process mining increasingly relies on object-centric event logs (OCELs) to capture complex process behaviours through the intersecting participation of varied objects. While OCELs remove the restrictive single-case assumption of traditional process mining, they introduce new complexities: the participation of objects in events carries far more ontological weight in defining process behaviour. Consequently, heuristic-driven abstraction and execution-identification methodologies suffer from structural flaws. While the flat data structure of case-centric mining naturally limits the compounding effect of these heuristic errors, the interconnected nature of object-centric process mining requires a more rigorous foundation.

Relying purely on structural and topological properties—such as connected components—to define process boundaries in OCELs obfuscates foundational process semantics and ignores the critical role of domain knowledge. Furthermore, existing methodologies for case determination are prone to over-merging errors and struggle to produce executions at multiple levels of granularity.

To address these gaps, this paper introduces a unified, declarative framework for event abstraction and case determination grounded in an applied theory of *mereotopology*—the logical study of parts, wholes, and connectedness. We demonstrate that event abstraction and case determination are fundamentally problems of composition. By formalizing object co-participation through logical axiom patterns, we define topological connections between events using reusable, parametric axioms. These axioms enable domain knowledge to explicitly dictate when co-participation constitutes a connection and how connected parts (events) sum into wholes (executions). As a result, both process executions and abstracted activities are defined as domain-governed sums of their constituent events. We show that translating these logical axioms into executable rules structurally precludes standard correctness issues, including convergence and divergence, by resolving them at the axiomatic level. Ultimately, this mereotopology-based approach provides a transparent, verifiable, and semantically sound foundation for defining process perspectives in object-centric process mining.

ACM Reference Format:

Riley Moher and Michael Gruninger. 2026. Through the Object-Centric Looking Glass: a Mereotopology-based Foundation for Transparent Event Log Abstraction and Case Extraction. 1, 1 (September 2026), 13 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Process mining bridges the gap between raw data and process reality. However, recorded events often reside at a lower level of abstraction than the analyst’s intended process description. To recover meaningful behaviour, these low-level events must be aggregated into larger, more complex units. Deciding how and where to group this data is not merely a data preparation step; it constitutes a significant ontological commitment regarding which entities compose the process being analyzed, and at what level of granularity.

While traditional case-centric mining trivializes this grouping through a case identifier, object-centric event logs (OCEL) intrinsically lack an analogous mechanism. Because OCELs record how objects of several types participate in events, a grouping based on one object type yields a completely different structure than a grouping based on another, or on the relations between them [11]. Consequently, the space of admissible aggregations is vast. Existing approaches to navigate this space remain immature, built for highly pragmatic analytical purposes, making them difficult to compare or reason about systematically.

Authors’ Contact Information: Riley Moher, riley.moher@mail.utoronto.ca; Michael Gruninger, University of Toronto, Toronto, Canada.

2026. Manuscript submitted to ACM

Manuscript submitted to ACM

Furthermore, the literature currently treats the aggregation of event data as two distinct problems. On one hand, event abstraction aggregates fine-grained events into higher-level activities [27]. On the other, case extraction determines which events constitute an individual process execution [26]. Both tasks fundamentally concern how smaller process entities constitute larger process wholes. While numerous techniques address these tasks individually [2, 26, 27], they lack a formal account of the underlying common problem. Specifically, they leave implicit what makes one process entity a part of another, and how process entities relate independently of their membership in a common whole. Because these answers are currently opaquely encoded into algorithms and heuristics, the theoretical basis of the resulting structures remains unstated, offering no clear mechanism to separate foundational principles from domain-specific knowledge.

These two missing relationships correspond directly to the central notions of *mereotopology*: mereology provides a formal theory of parthood, while topology provides a theory of connection. In the context of process data, parthood provides the basis for abstraction, formally describing how lower-level events act as parts of more complex process occurrences. Connection provides a basis for relating entities independently of their shared wholes; for example, in object-centric data, co-participation of objects in events offers a natural data-level interpretation of such connection. Together, these relations establish a common theoretical foundation for both event abstraction and case extraction, capable of handling overlapping structures and multiple levels of abstraction. This foundation separates what is general about process structure from what must be supplied by a particular application: foundational axioms specify the mereotopological theory, while domain knowledge determines how recorded events and objects instantiate that structure. Ultimately, this yields a unified, domain-extensible account of object-centric abstraction and case extraction.

We structure this work through the following contributions:

- (1) We provide a logically rigorous theory of object-centric event abstraction and case extraction based on mereotopology. We express the theory as a parameterized axiom schemata, so that a domain theory is a finite set of type-level declarations, and we characterize what domain knowledge contributes beyond a process model.
- (2) We establish the theory’s handling of standard correctness issues in the case extraction setting including convergence, divergence, and deficiency.
- (3) We present an implementation of the theory over benchmark OCEL 2.0 logs, demonstrating that a foundationally-grounded account of abstraction can be applied at realistic scale.

The paper is organized as follows. Section 2 situates the work against object-centric event logs, event abstraction, case extraction, and the relevant theoretical and ontological background. Section 3 develops the conceptual core against a running example. Section 4 gives the formalization and its correctness properties, Section 5 the implementation and validation, and Section 6 concludes.

2 Background and Related Work

2.1 Object-Centric Event Logs

OCPM as standardized by the OCEL and OCEL 2.0 standards formalize the object-centric event log structure [5, 11] and underpin a growing body of techniques [1]. In a case-centric log the trace is given by an identifier, whereas here neither the extent of a process instance nor the granularity of its activities is fixed by the data, so several case notions may be simultaneously valid and none is distinguished. Another ambiguity concerns the relationships among co-participating objects. These are frequently described as *synchronizations* [10, 12], but the term covers qualitatively

different phenomena: Petri-net-style token synchronization for conformance [12], stable relationships between co-occurring objects [22], and unobserved “silent” objects whose participation explains an observed co-participation [25]. These accounts make object relationships increasingly explicit, but remain oriented toward discovery, conformance checking, and modelling constructs rather than a general characterization of which co-participations are process-relevant and why. While OCEL 2.0 does enable the declaration of object-to-object (O2O) relations directly, these take the form of simple edges between individual object instances. These relationships take the form of instance-level data rather than foundational or domain theory, and thus lack generalizable information on what follows from them, particularly as it pertains to abstraction and case notions.

2.2 Event Abstraction

Event abstraction, transforming fine-grained event data into meaningful behaviour at a business level, has been studied extensively. Van Zelst et al. [27] survey and taxonomize techniques spanning supervised, unsupervised, and knowledge-based approaches, illustrating both the appeal and the limits of the state of the art: the space of techniques is diverse, yet it relies heavily on the simplifying assumptions of case-centric data in which one identifier partitions events into independent traces [16]. We explain three key limitations that follow from the reliance on these assumptions.

First, there is no account of what an abstraction is that is independent of the technique producing it. Techniques are specified operationally — clustering algorithms, optimization problems, pattern-matching rules — and compared on benchmark performance. There is no explicit commitment to what must be true of the events, objects, and relations involved for a grouping to be admissible at all, and without it the logical consequences of an abstraction (whether it is sound, whether two compose, whether one refines another) cannot be assessed.

Second, where domain knowledge is used, it enters as a parameter rather than as a theory. Senderovich et al. [23] bridge sensor data and process-level events via *interactions* as an intermediate knowledge layer, solving the abstraction problem as an integer linear program whose parameters encode existing process knowledge. Semantic-annotation style approaches such as Klessascheck et al. [18] generate high-level healthcare events by exploiting structured medical vocabulary as concept hierarchies. In both cases, while the knowledge being applied and results obtained are useful, the domain knowledge is opaquely encoded as ILP coefficients or a separate rules language encoding, neither available for reasoning *about* the abstraction. The encoding is consequently not generalizable: moving to a new domain means rebuilding the artifact, since it lacks a domain-independent form.

Third, techniques reliant on a case identifier do not simply transfer to the OCPM setting. With several participants per event there is no single trace to segment, and the decision of which events form a candidate group is inseparable from the decision of which objects individuate the resulting whole; the two collapse into one problem that the structure of the log cannot settle.

The solution should then be accordingly specific. The formal foundation is domain-independent, and the criterion for composing events into wholes is not fixed within it but declared: a logical theory parameterized by a domain’s object types and relations. Expressing domain knowledge in the core’s own language makes it inspectable, reusable, and verifiable against a candidate set of occurrences.

2.3 Object-Centric Case Extraction

Recent work has begun to formalize case extraction for object-centric logs directly. Adams et al. [2] define cases through connected components of the object interaction graph and prove that this construction avoids the issues of deficiency, convergence, and divergence. These structural anomalies occur when an event belongs to no case (deficiency), multiple

events of the same activity belong to a single case (convergence), or a single event is shared across multiple cases (divergence). While this is a useful guarantee, it introduces a meaningful risk of taking the most permissive grouping available, such that resource-like objects transitively fuse executions into cases that are not meaningful. Van Detten et al. [26] recognize this and develop a more general space of case notions, together with heuristics for navigating it.

Three limitations persist. First, heuristics over the participation graph can *suggest* but not *settle* whether an object type is a re-usable resource or a true identifier of process behaviour. This distinction fundamentally relies on domain knowledge; purely structural metrics cannot distinguish a log that naturally consists of a few massive cases—such as the multi-year assembly of a single commercial airplane—from a high-volume e-commerce log where thousands of independent customer orders have been erroneously grouped together because they used the same delivery truck. Second, because existing correctness results are stated in terms of algorithmic procedures, establishing guarantees for any variant relies a relatively difficult-to-obtain guarantee of program behaviour rather than verifying that a model satisfies a formal theory. Finally, these methods yield a single, flat case notion per execution, offering no formal account of how different levels of abstraction or parameterizations may relate to one another.

2.4 Mereology and Mereotopology

Mereology, the formal theory of the parthood relationship of parts and wholes, provides the foundation on which our theory is built. Classical mereology defines parthood as a relation with three properties:

- **Reflexivity:** Everything is part of itself. $\forall x \text{ part}(x, x)$
- **Antisymmetry:** No two distinct things are both part of each other. $\forall x, y \text{ part}(x, y) \wedge \text{part}(y, x) \implies (x = y)$
- **Transitivity:** If an object is part of another object, and that object is part of a larger object, the first object must be part of the last. $\forall x, y, z \text{ part}(x, y) \wedge \text{part}(y, z) \implies \text{part}(x, z)$

Cotnoir and Varzi [9] give a comprehensive and up-to-date account of mereology derived from these core axioms of classical mereology, their variants, and the philosophical and formal considerations that motivate departures from the classical axiomatization. To extend a purely mereological vocabulary of parts and wholes to a vocabulary capable of expressing spatial and relational structure, mereology is standardly supplemented with a primitive *connection* relation, yielding mereotopology, a theory of parts, wholes, and their boundaries and contacts. Casati and Varzi’s treatment of mereotopology [8] develops this combined framework in an accessible, tutorial-style manner, motivating the connection relation and showing how mereology and topology can be jointly axiomatized to support reasoning about parthood and contact together. We adopt this simple mereotopological theory - reflexive, antisymmetric, transitive parthood with a connection relation - as the formal background for characterizing event abstraction (parthood) through co-participation among objects in object-centric event logs (connection).

2.5 Process Ontologies

Foundational ontologies such as DOLCE [6] and UFO [15] provide formal accounts of occurrents, participation, and part-whole structure, but their expressiveness makes direct application to enterprise data difficult. While prior work has begun to address these issues for process mining [19], it lacks an object-centric focus and does not explicitly target process mereology.

While we do not make direct use of any existing process ontology, we adopt terminology from the ISO-standard Process Specification Language (PSL) ontology [13]. **Activities** are repeatable patterns of behaviour, **activity occurrences** are their concrete temporal realizations, and **objects** are continuants that **participate** in occurrences. Activity occurrences may be *atomic*, corresponding to individual indivisible events, or *complex*, composed of sub-occurrences. This

distinction allows both individual events and abstracted process executions to be treated within a common mereological framework.

3 Ontological Foundations of Object-Centric Process Abstraction

Composing lower-level elements into wholes requires a criterion of connection, which we call a *unity criterion*, and in the object-centric setting that criterion cannot be derived from structural properties of the log alone. This section develops the claim against a running example, identifies what knowledge supplies a unity criterion, and states the questions Section 4 must answer.

Running Example. Table 1 shows a fragment of an object-centric event log from a logistics setting. Orders (o_x) are registered with a transport document (d_x), for which a container (c_x) is then ordered; the container is loaded onto a truck (t_x) by a forklift (f_x), before being loaded onto a final departure vehicle (v_x).

Table 1. Example object-centric event logistics log snippet

Event ID	Activity	Timestamp	Involved Objects
e_1	Register Order	08:00	o_1, d_1
e_2	Order Container	08:30	d_1, c_1
e_3	Register Order	08:45	o_2, d_2
e_4	Order Container	08:55	d_2, c_2
e_5	Load onto Truck	09:00	c_1, t_1
e_6	Unload Container	11:00	c_1, t_1, f_1
e_7	Load onto Truck	13:00	c_2, t_1
e_8	Unload Container	15:00	c_2, t_1, f_1
e_9	Load Departure Vehicle	16:30	c_1, c_2, f_1, v_1

From Identity to Participation. Even this fragment leaves open which events belong together as parts of one execution: the two containers are distinct, yet share a truck and a forklift, and each is preceded by its own order and document. The simplest unity criterion is object identity, and c_1 's participation history e_2, e_5, e_6, e_9 already forms a coherent occurrence, its own object trace; this is essentially the case-centric setting, and every classical single-case-notion log is a degenerate instance of our framework in which one object type's identity is the only criterion. Extending it object-centrally involves more than admitting further types, because one object's lifecycle often does not describe the intended process. With Container as the case-defining type, e_9 involves both containers and is replicated across two occurrences: *convergence* [2, 11]. With Truck, every event involving t_1 forms one occurrence, collapsing two distinct shipments into one occurrence: *divergence* [2, 11]. Both issues reveal the gap between a process identity criterion and the semantics of an activity constituted by many participants.

Object lifecycles intersect at common events, commonly called "synchronization" [10, 12]; we use the more neutral *co-participation*. A *connection criterion* generalizes identity by specifying which co-participations between a pair of object types let their lifecycles connect, the transitive closure of connected events forming a whole. Declaring Document/Container and Order/Document connectable draws d_1 into c_1 's occurrence via e_2 and transitively o_1 via e_1 , so that the order's other events are drawn in even though it never co-participates with the container directly, capturing end-to-end fulfillment. The same criterion applied to Container and Forklift has the opposite effect, merging two distinct occurrences because f_1 co-participates with both. Skewed participation frequencies may heuristically suggest a type acts

as a resource [26], but the distinction cannot be guaranteed structurally. Criteria must therefore be declared pairwise rather than uniformly, and may be refined further, restricted to specific activities or conditioned on attributes.

Almost none of what settles the questions above is about ordering. It is knowledge about objects: that trucks are shared across shipments while containers are not, that a transport document is issued *for* an order whereas a forklift merely acts on a container, that a truck has room for one container at a time. None of this is recoverable from the log, which records only that the objects co-participated.

It is also what delineates the broad notion of synchronization. One co-participation relation stands for several different relationships: the propagation of identity from an order to its document, the use of a forklift as a shared resource, and at e_9 the joining of two executions in which no single container is “the” identifying container of the event. Domain knowledge separates these, and takes more than one form in doing so — a type being a resource, a pair of types being related, a relation holding only within a given activity, or a limit on how many objects may be jointly related.

From Foundational Core to Domain Knowledge. None of this requires a bespoke formalism per domain. Theories of mereology and mereotopology are widely studied at a grounded application level; for instance, Grüninger et al. demonstrate that a mereology requiring connected sums applies across manufacturing assemblies, molecular structures, and gene sequences [14]. Our theory shares this foundational ontological commitment. We rely on a domain-independent mereotopological core, while domain-specific knowledge acts as an extension to specify connection criteria. Rooting the identification of complex activity occurrences in this general logical theory makes the approach highly adaptable to expressive domain extensions. Returning to our running example, consider the physical constraint that a truck can hold only a single container. With a geospatial extension, the theory could directly encode co-location constraints—ensuring, for example, that objects remain co-located until an explicit separating event occurs (as has been demonstrated extensively in [4]). Because such elaborations build on a single, reusable core theory, they can be introduced without rebuilding the formalism from scratch, providing a stable foundation for the implementation strategies discussed later.

Within this ontologically-grounded paradigm, the three correctness issues usually invoked when a case notion is chosen are not defects to be eliminated but questions a domain theory must answer explicitly. *Deficiency* concerns atomic occurrences, since every event is reflexively connected to itself and the relevant notion is not absence of connection but the existence of atomic occurrences, parts that belong to no larger whole. *Convergence* is ontologically permissible through the overlaps relation, and this result is unremarkable within applied mereotopology [14]; its difficulties arise downstream, when durations are aggregated or activities counted. While some previous work has dealt with handling ontology-based preconditions for such operations [20], it is outside of the scope of this paper. *Divergence* is a scope decision about how far an occurrence extends. Formulating these issues as axioms makes satisfaction a finite model check and establishes the consequences of a declaration once, at the level of the theory, rather than per criterion; Section 4.4 does so for all three. Domain knowledge is therefore not an optional refinement to structural methods but a prerequisite, since the missing events and quality issues endemic to real logs prevent purely structural approaches from recovering the intended semantics on their own.

4 Formalization of Process Connections, Parts, and Wholes

An object-centric event log is conventionally specified as a tuple $L = (E, O, OT, act, obj, otype, \prec)$ over finite sets of events E , objects O , and object types OT , recording for each event its activity (act), its participating objects (obj), each

object's type (*otype*), and a total temporal order $\prec$ on E , in the style of [10, 24]. We take as given a standard translation of L into ground facts of a first-order knowledge base¹ over a simple signature of relations:

$occurrence(o)$	o is an activity occurrence (atomic or complex).
$occurrence_of(o, a)$	o is an occurrence of activity a .
$object(x)$	x is an object.
$T(x)$	x is an object of type T , for each $T \in OT$.
$participates(x, o)$	object x participates in occurrence o .

Object types satisfy domain closure: $\forall x (object(x) \equiv \bigvee_{T \in OT} T(x))$, together with pairwise disjointness, matching *otype*'s assignment of a unique type to each object. The translation of L itself is direct: every $e \in E$ yields $occurrence(e)$ and $occurrence_of(e, act(e))$; every $o \in O$ yields $object(o)$ and $T(o)$ for $T = otype(o)$; and every $o \in obj(e)$ yields $participates(o, e)$. For each object o , its *object trace* $Tr(o) = \langle e_1, \dots, e_k \rangle$ is the $\prec$ -ordered sequence of events with $participates(o, e_i)$; on the running example, $Tr(c_1) = \langle e_2, e_5, e_6, e_9 \rangle$. We build the remainder of the theory of connectedness and parthood in terms of this logical signature.

Axiom Schemata and Domain Theories. Everything that follows is stated as *axiom schemata*: domain-independent templates carrying formal parameters that range over object types and activities, similarly to function definitions in programming languages. A schema is not itself an assertion about any particular domain; it becomes one when its parameters are supplied, expanding into a closed set of first-order sentences. As a simple example, a schema $Inside(X, Y)$ may state that every region of type X lies within some region of type Y ; setting $X = City$ and $Y = Country$ yields the axiom that every city lies within a country. A *domain theory* is a finite list of such declarations over the fixed axioms of Section 4.1, so writing one requires no logic.

4.1 A Mereotopology of Connection

The theory is built from named component theories – ground mereology with *part* (part of) as a reflexive, anti-symmetric, and transitive relation, and *C* (connection) a reflexive, symmetric relation with axioms for overlaps, weak supplementation, and upper bounds². Two definitional extensions of parthood are useful throughout: $atomic_part(x) \equiv \forall y (part(y, x) \rightarrow y = x)$, which in our case are events, $event(x) \leftrightarrow occurrence(x) \wedge \neg \exists y subocc(y, x) \wedge x \neq y$, as well as its complement $complex(x) \equiv \exists y (part(y, x) \wedge y \neq x)$, identified as complex activity occurrences.

What matters for grounding co-participation is how *part* and *C* interact, and how *C* constrains sums. We axiomatize notable properties of our general mereotopology as follows:

Whatever a part is connected to, the whole is connected to:

$$\forall x, y (part(x, y) \rightarrow \forall z (C(z, x) \rightarrow C(z, y))) \quad (\text{Mono})$$

A sum is connected to exactly what its summands are:

$$\forall x, y, s (sum(x, y, s) \rightarrow \forall u (C(u, s) \leftrightarrow C(u, x) \vee C(u, y))) \quad (\text{SumC})$$

Summability requires connection:

$$\forall x, y (\exists s sum(x, y, s) \rightarrow C(x, y)) \quad (\text{Sum} \rightarrow C)$$

¹Pragmatically this translation is not free; in our implementation it takes the form of a virtual knowledge graph mapping.

²The axiomatization for this baseline theory is available at github.com/gruninger/colore/blob/master/ontologies/combined_mereotopology/weak_cisco_mt.clif

Sum is functional

$$\forall x, y, s_1, s_2 \ (sum(x, y, s_1) \wedge sum(x, y, s_2) \rightarrow (s_1 = s_2)) \quad (\text{SumU})$$

(Mono) and (SumC) are what make the theory *mereotopological*: an occurrence’s connectivity is fixed entirely by the connectivity of what composes it. (SumU) makes summation functional, so that any two elements have at most one sum. This may seem trivial in the applied setting of process mining, but it is not a default across theories of mereotopology, and we state it explicitly because the fusion schema of Section 4.3 relies on it.

(Sum $\rightarrow$ C) is the weaker, one-directional form of an axiom that some mereotopologies state as an equivalence; we instead deliberately assert the implication rather than the bidirectional. This equivalence is well motivated for some domains—such as assembly of parts—where a connected aggregate simply is a whole, but it is a specific commitment rather than a general feature of mereotopologies, many of which leave sum existence unrestricted or govern it independently of connection [14]. For process mining, this leads to runaway sums, since events shared between processes and resources participating in many events connect nearly everything to nearly everything else [26]. We therefore substitute the sufficiency direction with a domain-conditioned sum existence theory, detailed in Section 4.3.

4.2 Grounding Connection in Co-Participation

Connection holds between *events*, as determined by connection criteria that may involve their activities and their shared participants. We specify these criteria through two axiom schemata, both stated in terms of co-participation of objects in an event.

The first schema is the simple case of connection through a *shared individual*: for a selected type T in the domain theory (for instance, containers in the running example), an object of type T participating in two events connects them.

$$\forall e_1, e_2, x \ (T(x) \wedge participates(x, e_1) \wedge participates(x, e_2)) \rightarrow C(e_1, e_2) \quad (\text{IdConn}(T))$$

This is deliberately not asserted for every type by default. Since object types are pairwise disjoint, (IdConn) holds only for a shared individual of type T , never for two distinct objects sharing a type, so asserting it for $T = \text{Container}$ keeps $Tr(c_1)$ and $Tr(c_2)$ separate, while declining to instantiate it for *Forklift* or *Truck* avoids imposing a resource view on a process description about order fulfillment.

The second schema establishes a connection through *co-participation* with a declared pair of distinct types:

$$\begin{aligned} \forall e_1, e_2 \left(\exists o_1, o_2, e_{co} \left(T_1(o_1) \wedge T_2(o_2) \wedge participates(o_1, e_{co}) \wedge participates(o_2, e_{co}) \wedge participates(o_1, e_1) \right. \right. \\ \left. \left. \wedge participates(o_2, e_2) \wedge e_1 \neq e_2 \right) \rightarrow C(e_1, e_2) \right) \quad (\text{TypeConn}(T_1, T_2)) \end{aligned}$$

It may be strengthened so that T_1 and T_2 are connected only through co-participation in a given activity a , by adding the conjunct $occ_of(e_{co}, a)$ to the antecedent. We write this third schema (ActConn(T_1, T_2, a)).

Instantiating these schemata is how a domain theory specifies which objects form connected wholes. Declaring $TypeConn(\text{Document}, \text{Container})$ and $TypeConn(\text{Order}, \text{Document})$ together with $IdConn(T)$ for $T \in \{\text{Order}, \text{Document}, \text{Container}\}$, exactly captures the order-fulfillment process description of the running example as described in Section 3. Declaring $ActConn(\text{Forklift}, \text{Vehicle}, \text{“Load Departure Vehicle”})$ instead recovers the activity-conditioned refinement. This also makes precise the relationship to O2O relations in OCEL 2.0 [5]: an O2O relation is a ground fact about one pair of objects, whereas these are universally quantified axioms over events that propagate across the whole log via (Mono), rather than requiring each connection to be asserted individually.

We write $\mathfrak{B}$ for the set of types for which (IdConn_T) has been instantiated in a given domain theory, and call these the *identity-bearing* types. In the order-fulfillment theory, $\mathfrak{B} = \{\text{Order}, \text{Document}, \text{Container}\}$. Trucks, forklifts and vehicles are resources, which may still connect events through other axioms in the domain theory such as instances of (Act-Conn) but never carry identity.

4.3 Domain Conditions on the Existence of Sums

Connection is only one element of a mereotopology, and we now focus on the conditions for which those connections have a valid sum, forming a connected whole. Summing over any set of connected elements identifies misleading occurrences. In the running example the “Load Departure Vehicle” occurrence e_9 has two containers as participants, c_1 and c_2 ; unrestricted summation therefore yields an upper bound over both of their object traces, or at worst fuses the entire log, obfuscating the two order fulfillments we set out to identify. Just as connections are established by domain-specific criteria, sums exist only under *domain conditions*, given again by axiom patterns instantiated in the domain theory.

The first condition is an identity criterion on occurrences. An event e is *determinate* with respect to an identity-bearing type $T \in \mathfrak{B}$ when at most one object of that type participates in it:

$$(\forall e \text{ det}_T(e) \equiv (\forall x, y T(x) \wedge \text{participates}(x, e) \wedge T(y) \wedge \text{participates}(y, e)) \rightarrow (x = y)) \quad (\text{Det})$$

Resource types are exempt by construction (since no resource type $\in \mathfrak{B}$), so the co-participation of two forklifts in one event is immaterial. In the departure event e_9 container identity is *indeterminate*: it has no container in the sense required to propagate identity, being a synchronization point at which “the” container of the occurrence is undefined³.

Two schemata and one closure axiom then determine which sums exist:

Each object of a declared type generates a whole composed of the object traces of everything that generates it, and of nothing else:

$$\forall o, e \left((T(o) \wedge \text{event}(e)) \rightarrow (\text{part}(e, \sigma(o)) \leftrightarrow \exists o' (\sigma(o') = \sigma(o) \wedge \text{participates}(o', e))) \right) \quad (\text{Sum}(T))$$

Determinate co-participation makes two objects generate the same whole:

$$\begin{aligned} \forall o_1, o_2, e \left((T_1(o_1) \wedge T_2(o_2) \wedge \text{participates}(o_1, e) \wedge \text{participates}(o_2, e) \wedge \text{det}_{T_1}(e) \wedge \text{det}_{T_2}(e)) \right. \\ \left. \rightarrow \sigma(o_1) = \sigma(o_2) \right) \quad (\text{Fuse}(T_1, T_2)) \end{aligned}$$

No complex occurrences exist beyond those so generated:

$$\forall z \left(\text{complex}(z) \rightarrow \exists o \left(\bigvee_{T \in \mathfrak{B}} T(o) \wedge z = \sigma(o) \right) \right) \quad (\text{Only})$$

Since summation is functional by (SumU) , (Fuse) is stated as an identity between wholes rather than as a further act of summation. The two schemata fix σ jointly and we take the least assignment satisfying them⁴. (Sum) is the existence principle replacing the sufficiency direction dropped in Section 4.1, and (Only) its closure clause: connection remains necessary for summability by $(\text{Sum} \rightarrow \text{C})$, while sufficiency is conferred only by a generator.

Declaring $\text{Sum}(T)$ for $T \in \mathfrak{B}$ with $\text{Fuse}(\text{Order}, \text{Document})$ and $\text{Fuse}(\text{Document}, \text{Container})$ captures the intended description (Figure 1): an order and a document participate determinately in e_1 , a document and a container in e_2 , and no instance of (Fuse) applies at e_9 , which is not $\text{det}_{\text{Container}}$, so six identity-bearing objects generate two occurrences rather than six. The wholes *overlap* at e_9 and are connected by (Mono) , but (Sum) and (Only) permit no third occurrence

³In this case, the process mining terminology of “synchronization” serves a more appropriate and intuitive function as a subclass of co-participation.

⁴Read as a simultaneous inductive definition, this is the theory’s only such component; in the implementation it is a Datalog fixpoint.

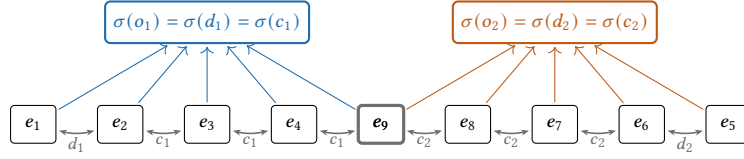


Fig. 1. The mereotopology induced by $Sum(T)$ for $T \in \mathfrak{B} = \{Container, Order, Document\}$ with $Fuse(Order, Document)$ and $Fuse(Document, Container)$. Vertical edges denote parthood, while horizontal edges denote connection.

composed of them. The declaration of $TypeConn(T_1, T_2)$ instantiates $Fuse(T_1, T_2)$ whenever $T_1, T_2 \in \mathfrak{B}$, and $Sum(T)$ is instantiated for every $T \in \mathfrak{B}$. A domain theory therefore declares only which types are identity-bearing and which pairs co-participate; the sum-existence axioms follow from those declarations, in the manner of one call expanding into several. This also yields multiple levels of abstraction rather than a single homogeneous case notion: adding *Truck* to $\mathfrak{B}$ and declaring $TypeConn(Container, Truck)$ identifies $\sigma(t_1)$ with both container wholes and returns the whole log as one fulfillment occurrence, as expected owing to the truck's resource-like utilization.

4.4 Correctness Properties

We previously framed deficiency, convergence, and divergence as questions a domain theory must answer rather than defects to be eliminated. We now address these issues as consequences of the schemata, established once and inherited by every domain theory that instantiates them, rather than as arguments about some procedure of abstraction⁵.

Everything follows from how (Fuse) behaves. Each of its instances identifies two identity-bearing objects that co-participate determinately in some event. Chaining these identifications groups the identity-bearing objects together, and since identity is an equivalence relation, the groups partition them.

LEMMA 4.1. *Two identity-bearing objects generate the same whole exactly when a chain of (Fuse) instances links them, and each whole contains exactly the events in which some member of its group participates.*

PROOF SKETCH. (Fuse) gives one direction directly. For the other, assign each identity-bearing object to its group, and let an event be part of a whole whenever some member of that group participates in it. This satisfies (Fuse) by construction and (Sum(T)) by its definition of parthood. As σ is the least assignment satisfying both schemata, it identifies no more. $\square$

Deficiency. Deficiency is characterized by an event that is part of no generated whole, $Deficient(e) \equiv \neg \exists z (part(e, \sigma(z)))$. By (Sum(T)), an event is part of $\sigma(z)$ only if some identity-bearing object participates in it, and conversely any identity-bearing participant z places the event in $\sigma(z)$. Deficient events are therefore exactly those whose participants are all non-identity-bearing. Each deficient event admits two conclusions: either the domain theory is under-specified with respect to the event log, or the event log records behaviour outside the process the theory describes. In the first case, some type participating in the deficient event should have been declared identity-bearing. In the second, the event is correctly excluded.

Convergence. An event lies in two wholes exactly when two identity-bearing objects from different groups participate in it, $Convergent(e) \equiv \exists x, y (part(e, \sigma(x)) \wedge part(e, \sigma(y)) \wedge \sigma(x) \neq \sigma(y))$. So where every identity-bearing type present in an event is determinate and the relevant pairs are declared, (Fuse) puts those objects in one group and the event lies

⁵While here we abbreviate the proofs, the complete, formal and machine-verifiable proofs are provided in <https://github.com/riley-momo/ThroughTheObjectCentricLookingGlass/tree/main/proofs>

in a single whole. Convergence is not eliminated, as a shared part between two wholes is theoretically consistent in the mereotopology, and is instead confined to events for which some identity-bearing type is indeterminate. In the running example the only such event is e_9 : two containers participate, so it is not $det_{Container}$, no instance of (Fuse) applies, and it remains a shared part of two distinct wholes rather than forcing them into one.

Divergence. Declaring a further *TypeConn* pair adds chains without removing any, so it can only merge groups, never split them, and which wholes merge is known before evaluation rather than discovered in the output. Divergence, as ordinarily understood, is the complaint that a grouping so obtained contains too much: declaring *Truck* identity-bearing together with *TypeConn(Container, Truck)* collects the events of several containers into one occurrence, and whatever ordering was expected of a single container is lost. This is not a defect in the extraction but a mismatch between the intended semantics of a domain theory and the one actually declared, identified and corrected at the meta-level by revising the declaration. The usual framing also carries assumptions worth making explicit. Adams et al. [2] take deficiency to imply divergence, on the grounds that missing events leave missing precedence constraints. This presupposes that divergence is judged against a control-flow model whose ordering constraints hold between activities irrespective of the objects involved, and that those constraints are total, so that every event label in the log must be ordered. Our formulation requires neither.

5 Implementation and Validation

The implementation⁶ is a two-stage pipeline. Event log data remains in its original format, and a virtual knowledge graph built with Ontop [3] maps it to ground facts, which is memory-efficient and scalable but limited in reasoning power. A hybrid approach alleviates this: simple mapping logic runs in the virtual knowledge graph, and when queries are issued over it the necessary facts are materialized and loaded into a more capable reasoner, here the Soufflé Datalog compiler [17]. Using a virtual knowledge graph for process mining follows established practice [7], keeping the mapping an inspectable artifact rather than an extraction script, leaving the database backend free, and avoiding an upfront materialization.

Neither stage is intended to be visible to the practitioner. Because every schema of Sections 4.2–4.3 is parameterized by object types alone, a domain theory contains no quantifiers, variables, or axioms for a user to write: it is fixed by two choices, which types are *focal*, fixing $\mathfrak{B}$, and how those types are connected, with activity-restricted variants offered as refinements. What a domain expert supplies is therefore close to the minimum anyone attempting this analysis must already know: which objects the process is about, and what relates them. Alternative process descriptions are entered by revising declarations rather than rewriting extraction code, with Section 4.4 fixing in advance how the resulting occurrences relate to the previous ones.

Dataset	$ E $	$ O $	$ OT $	$ F $	Occurrence Count	Time (s)
Order Management	21 008	10 840	6	179 237	2 000	0.88
Container Logistics	35 761	14 013	7	123 530	600	0.17
Procure-to-Pay	14 671	9 543	10	74 337	927	0.40
BPI 2017 [21]	23 561	5 883	4	86 111	1 575	0.98

Table 2. Evaluation times for identifying occurrences in benchmark OCEL 2.0 logs. $|E|$ denotes number of events, $|O|$ objects, $|OT|$ object types, $|F|$ the input knowledge base size (count of literals).

⁶Implementation available at <https://github.com/riley-momo/ThroughTheObjectCentricLookingGlass>

The implementation was run against several benchmark OCEL 2.0 datasets⁷. Table 2 reports, for each, the size of the log, the number of complex occurrences σ identifies under a domain theory declaring focal types and connections aligned with the dataset’s own process description, and the evaluation time. The intent is a proof of concept, establishing that the schemata evaluate over logs of realistic size on consumer-grade hardware⁸ in reasonable time. A formally grounded approach can therefore also be a tractable one: the declarations are axioms with a model-theoretic reading, yet remain performant because the cost is borne by a database engine and a fixpoint computation rather than by general-purpose theorem proving.

The hybrid architecture also recovers expressiveness lacking in both standard Datalog and Description Logics: Datalog cannot infer new entities at all, and DLs restrict inference to entities linked to a single parent object, whereas identifying complex occurrences requires generating an entity from the interaction of several objects, as in $\forall x, y, z \Phi(x, y, z) \rightarrow \exists o \Theta(o)$. The mapping layer bridges this gap, with a SPARQL query retrieving the multi-variable bindings satisfying Φ and the mapping constructing o as a deterministic function of them; (Sum(T)) uses exactly this mechanism to introduce occurrences that do not exist in the raw log. The trade-off is structural: offloading object creation to the mapping splits process logic across two artifacts, so debugging inconsistencies requires the wrapper to trace the provenance of Datalog entities back to their source bindings in the VKG.

6 Conclusion and Future Work

This work establishes a mereotopological framework in which the wholes of a process—its cases and executions—are not statically given by an event log, but declared. Grounding connection in object co-participation and conditioning sum existence on domain knowledge detaches abstraction from ad-hoc extraction scripts, and the mereotopology presented here, a weakened analogue of CISCO [14] that drops its biconditional between connection and summability, allows complex occurrences to share sub-occurrences without collapsing into a single occurrence spanning the entire log. Convergence is thereby accommodated rather than avoided: an event in which objects of two executions participate is a common part of both, which the theory admits directly instead of treating as an error to be repaired. Deficiency and divergence are located in the same way, as consequences of the theory rather than properties of an implementation. The result is an executable theory parameterized by a few type-level declarations, letting analysts shift abstraction levels by explicitly declaring what their intended process is about.

Parameterized axiom schemata and hybrid reasoning architectures have been explored in parallel for broader domain knowledge [19]; we apply similar mechanisms to process mining specifically. A natural next step is formal alignment with PSL [13], from which our terminology is already drawn, and with it a schema for the nested complex occurrences the present flat schemata do not provide. PSL in full is undecidable, but its modular architecture admits a weaker subtheory, letting domain theories inherit established ontological commitments while remaining tractable.

Practical directions include the declaration interface, incremental evaluation, and industrial case studies. Because the schemata are parameterized by types, candidate instantiations can also be derived inductively from a log, for instance by analyzing the frequency of determinate co-participation between types. This turns the discovery of case notions into a search over schema instantiations, for which the theory can supply both the hypothesis space and the criteria.

References

- [1] Jan Niklas Adams. 2025. *Object-centric process mining: concepts and advancements*. Ph.D. Dissertation. Dissertation, RWTH Aachen University, 2025.

⁷<https://ocel-standard.org/event-logs/overview/>

⁸A machine with a Ryzen 7 255 and 32 GB of DDR5 memory.

- [2] Jan Niklas Adams and Wil M. P. van der Aalst. 2024. Addressing Convergence, Divergence, and Deficiency Issues. In *Business Process Management Workshops (BPM 2023 Workshops) (Lecture Notes in Business Information Processing, Vol. 492)*, Jochen De Weerd and Luise Pufahl (Eds.). Springer, Cham, 496–507. doi:10.1007/978-3-031-50974-2_37
- [3] Timea Bagosi, Diego Calvanese, Josef Hardi, Sarah Komla-Ebri, Davide Lanti, Martin Rezk, Mariano Rodriguez-Muro, Mindaugas Slusnys, and Guohui Xiao. 2014. The ontop framework for ontology based data access. In *The Semantic Web and Web Science: 8th Chinese Conference, CSWS 2014, Wuhan, China, August 8-12, 2014, Revised Selected Papers 8*. Springer, 67–77.
- [4] Yunpiao Bai. 2025. *A Spatiotemporal Ontology for Semantic Trajectories*. Ph. D. Dissertation. University of Toronto (Canada).
- [5] Alessandro Berti, István Koren, Jan Niklas Adams, Gyunam Park, Benedikt Knopp, Nina Graves, Majid Rafiei, Lukas Liß, Leah Tacke Genannt Unterberg, Yisong Zhang, Christopher Schwanen, Marco Pegoraro, and Wil M. P. van der Aalst. 2024. *OCEL (Object-Centric Event Log) 2.0 Specification*. Technical Report. Chair of Process and Data Science, RWTH Aachen University. Version 2.0. arXiv:2403.01975 [cs.DB]
- [6] Stefano Borgo, Roberta Ferrario, Aldo Gangemi, Nicola Guarino, Claudio Masolo, Daniele Porello, Emilio M. Sanfilippo, and Laure Vieu. 2021. DOLCE: A Descriptive Ontology for Linguistic and Cognitive Engineering. *Applied Ontology* 17, 1 (2021), 45–69. doi:10.3233/AO-210259
- [7] Diego Calvanese, Tahir Emre Kalayci, Marco Montali, and Stefano Tinella. 2017. Ontology-based data access for extracting event logs from legacy data: the onprom tool and methodology. In *International Conference on Business Information Systems*. Springer, 220–236.
- [8] Roberto Casati and Achille C. Varzi. 1999. *Parts and Places: The Structures of Spatial Representation*. MIT Press, Cambridge, MA.
- [9] A. J. Cotnoir and Achille C. Varzi. 2021. *Mereology*. Oxford University Press, Oxford.
- [10] Dirk Fahland. 2022. Process mining over multiple behavioral dimensions with event knowledge graphs. In *Process mining handbook*. Springer, 274–319.
- [11] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M. P. van der Aalst. 2021. OCEL: A Standard for Object-Centric Event Logs. In *New Trends in Database and Information Systems (ADBIS 2021 Short Papers, Doctoral Consortium and Workshops) (Communications in Computer and Information Science, Vol. 1450)*, Ladjel Bellatreche, Marlon Dumas, Panagiotis Karras, Raimundas Matulevičius, Ahmed Awad, Matthias Weidlich, Mirjana Ivanović, and Olaf Hartig (Eds.). Springer, Cham, 169–175. doi:10.1007/978-3-030-85082-1_16
- [12] Alessandro Gianola, Marco Montali, and Sarah Winkler. 2024. Object-Centric Conformance Alignments with Synchronization. In *Advanced Information Systems Engineering (CAiSE 2024) (Lecture Notes in Computer Science, Vol. 14663)*. Springer, Cham, 3–19. doi:10.1007/978-3-031-61057-8_1
- [13] Michael Grüninger. 2004. Ontology of the process specification language. In *Handbook on ontologies*. Springer, 575–592.
- [14] Michael Grüninger, Carmen Chui, Yi Ru, and Jona Thai. 2020. A mereology for connected structures. In *Formal Ontology in Information Systems: Proceedings of the 11th International Conference (FOIS 2020)*. SAGE Publications 1 Oliver’s Yard, 55 City Road, London, EC1Y 1SP, 171–185.
- [15] Giancarlo Guizzardi and Gerd Wagner. 2004. A Unified Foundational Ontology and Some Applications of it in Business Modeling. In *Proceedings of the CAiSE’04 Workshops*.
- [16] Christian W Gunther and HMW Verbeek. 2014. Xes-standard definition. (2014).
- [17] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On synthesis of program analyzers. In *International Conference on Computer Aided Verification*. Springer, 422–430.
- [18] Finn Klessascheck, Tom Lichtenstein, Martin Meier, Simon Remy, Jan-Philipp Sachs, Luise Pufahl, Riccardo Miotto, Erwin Böttinger, and Mathias Weske. 2021. Domain-Specific Event Abstraction. In *24th International Conference on Business Information Systems (BIS 2021)*, Witold Abramowicz, Sören Auer, and Elzbieta Lewanska (Eds.). Hannover, Germany, 117–126. doi:10.52825/bis.v1i.39
- [19] Riley Moher and Michael Gruninger. 2025. Process Representation Meets Operational Realization: An Architecture for Data-Driven Process Ontology Application Through Process Mining. In *Formal Ontology in Information Systems*. IOS Press, 196–209.
- [20] Riley John-Anthony Moher. 2022. *Meaningful Datatypes: Ontologically-Sound Dependent Type Systems for Data Science*. Master’s thesis. University of Toronto (Canada).
- [21] Elizaveta Povalyaeva, Ismail Khamitov, and Artyom Fomenko. 2017. Bpic 2017: density analysis of the interaction with clients. *BPI Challenge 2017* (2017).
- [22] Anjo Seidel, Simon Winkler, Alessandro Gianola, Marco Montali, and Mathias Weske. 2025. To Bind or Not to Bind? Discovering Stable Relationships in Object-Centric Processes. In *Conceptual Modeling (ER 2025) (Lecture Notes in Computer Science, Vol. 16189)*, Dominik Bork, Roman Lukyanenko, Shazia Sadiq, Ladjel Bellatreche, and Oscar Pastor (Eds.). Springer, Cham, 223–241. doi:10.1007/978-3-032-08623-5_12
- [23] Arik Senderovich, Andreas Rogge-Solti, Avigdor Gal, Jan Mendling, and Avishai Mandelbaum. 2016. The ROAD from Sensor Data to Process Instances via Interaction Mining. In *Advanced Information Systems Engineering (CAiSE 2016) (Lecture Notes in Computer Science, Vol. 9694)*, Selmin Nurcan, Pnina Soffer, Marko Bajec, and Johann Eder (Eds.). Springer, Cham, 257–273. doi:10.1007/978-3-319-39696-5_16
- [24] Wil MP Van Der Aalst, Guangming Li, and Marco Montali. 2017. Object-centric behavioral constraints. *arXiv preprint arXiv:1703.05740* (2017).
- [25] Jan Niklas van Detten, Pol Schumacher, and Sander J. J. Leemans. 2024. Object Synchronizations and Specializations with Silent Objects in Object-Centric Petri Nets. In *Business Process Management (BPM 2024) (Lecture Notes in Computer Science, Vol. 14940)*, Andrea Marrella, Manuel Resinas, Mieke Jans, and Michael Rosemann (Eds.). Springer, Cham, 57–74. doi:10.1007/978-3-031-70396-6_4
- [26] Jan Niklas van Detten, Pol Schumacher, and Sander J. J. Leemans. 2025. A Framework for Advanced Case Notions in Object-Centric Process Mining. In *Advanced Information Systems Engineering (CAiSE 2025)*. Springer, Cham. doi:10.1007/978-3-031-82225-4_30
- [27] Sebastiaan J. van Zelst, Felix Mannhardt, Massimiliano de Leoni, and Agnes Koschmider. 2021. Event Abstraction in Process Mining: Literature Review and Taxonomy. *Granular Computing* 6, 3 (2021), 719–736. doi:10.1007/s41066-020-00226-2